

Assignment 3: MNIST Handwritten Digit Classifier

Introduction to AI, Instructor: Liaw Rung Tzuo | 廖容佐

Antony Wilson | 魏彥晨 (412262319)

May 27, 2025

1 Problem Definition

The assignment is to implement different classifying agents that aim to classify handwritten digits of the **MNIST** dataset. The dataset provides 60 000 training images and 10 000 test images. The agent must use the training images to train or initialize the classifier. The test images are samples not included in the training data used for testing the agent's ability to classify novel inputs. The **Macro F1 score**, defined as

$$\text{MacroF}_1 = \frac{1}{m} \sum_i \frac{2 \cdot \text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

is the metric for measuring the accuracy of an implementation. The methods implemented are *decision tree*, *linear classifier*, *nearest neighbor*, and improvements on those methods.

2 Implementation

2.1 Decision Tree

I implemented the decision tree as a binary tree. Its children classify a certain condition as being satisfied or not. The condition is a threshold on the value of a specific pixel within the image. An image is classified when a leaf node is reached. Every leaf node has a label (0-9) assigned to it. The decision tree is generated in one greedy iteration by recursively selecting an attribute that results in an optimal split of training examples. The quality of the split depends on the entropy or impurity of the resulting example subsets. I implemented the **Gini coefficient** for measuring impurity as it is computationally less expensive than calculating entropy:

$$\text{Gini} = 1 - \sum_{i=1} p_i^2$$

A node in the tree is marked as a leaf, and splitting stops when one of the following criteria is met:

- The tree has reached **maximum depth**.
- The node's number of samples goes below the **minimum sample threshold**.
- The leaves it would generate offer less than the **minimum impurity reduction**.

These three parameters affect the structure of the final tree. Maximum depth is a general limitation to avoid overly deep trees. Having a minimum sample threshold and requiring a reasonable impurity reduction prunes the tree to prevent overfitting. In my tests, `MAX_DEPTH = 15`, `MIN_SAMPLES_SPLIT = 100`, and `MIN_IMPURITY_REDUCTION = 0.02` gave good results. I had to implement several optimizations, such as using *histograms* for keeping count of distributions in example subsets to make iterating over the high number of possible attributes computable (784×255 for up to 60 000 samples¹).

2.2 Linear Classifier

For implementing the linear classifier I used **softmax regression**. Each image is represented as a 785-dimensional vector. 784 values are the normalized pixel values and the 785th value is a bias term of 1.0. For each class (0–9), the model learns a corresponding weight vector. Classification is done by computing the dot product between the input and each class weight, giving a score per class. The softmax function turns the scores into probabilities adding up to 1.0. The predicted label is the one with the highest probability.

Training minimizes the **cross-entropy** loss between the predicted probabilities and the true label. I added an **L2 regularization** term to the loss to reduce overfitting and help with noise:

$$Loss = -\log(p_{\text{true}}) + \lambda \|W\|^2$$

Gradients are computed across **mini-batches** and used to update the weights with gradient descent. The softmax output is reused to compute the error signal for backpropagation. I implemented weight initialization using Gaussian noise with mean 0 and standard deviation 0.1, which helped prevent symmetry.

Parameters used:

- `LEARNING_RATE = 0.05`
- `BATCH_SIZE = 256`
- `EPOCHS = 15`

¹The data provided for this assignment is actually a modified version of MNIST, with only 780 pixels per image, but in favor of avoiding confusion I will write 784 throughout this report.

- LAMBDA = 0.0001

The weights are updated as:

$$W \leftarrow W - \eta \cdot (\text{gradient} + 2\lambda W)$$

Each epoch reshuffles the data and splits it into batches. Gradients are averaged across each batch before the weights are updated. The softmax implementation subtracts the max score before exponentiation for numerical stability. I experimented with decaying the learning rate after each epoch, but the effect was not noticeable in my tests so I kept it constant.

2.3 Nearest Neighbor

An agent using nearest neighbor classification differs from the other methods as there is practically no setup required. All training images reside in memory. The algorithm classifies an image by giving it the label of the training image with the shortest Euclidean distance to it. As only comparisons matter, I was able to use the squared distance, avoiding root computations:

$$\text{Distance}^2(q, x) = \sum_j (q_j - x_j)^2$$

Calculating even the squared distance from one query image to every other training image is computationally expensive and slow. Precomputing the normals can reduce the number of calculations per comparison. However, in my tests, abandoning comparisons early if their distance exceeds that of the current best comparison showed a greater increase in performance. These two optimizations don't work together, as precomputing normals changes the distance calculation in a way that makes it impossible to prematurely see a lower bound. For classifying all training and test images, I implemented threading. Query images are split across `numThreads` workers (which defaults to CPU hardware concurrency). An `atomic` index hands out work items. Progress messages are written from the threads with a mutex to keep the log readable. The only parameter is the thread count `numThreads = std::thread::hardware_concurrency()`.

3 Performance Comparison

The three methods implemented approach classification very differently. Each comes with advantages and disadvantages. Measurements can be seen in Table 1. The decision tree and linear classifier spend a lot of time setting up or training the agent, which results in faster classifications once they are ready. Using nearest neighbor, on the other hand, involves no training whatsoever and relies on iterating over all test data for every classification. The resulting runtime at classification is, therefore, unreasonably high. Classifying the 60 000 training images took 30 minutes, with 12 threads solving in parallel. Classifying with the decision tree is by far the quickest, taking less than 100 ms to classify

Implementation	Input	Macro F1	Classification Time (ms)	Initialization Time (ms) ²
Decision tree	Train	0.8616	44	130 941
	Test	0.8422	6	
Linear classifier	Train	0.9076	5 743	268 298
	Test	0.9128	957	
Nearest neighbor	Train	1.0000	2 463 501	0
	Test	0.9688	524 717	

Table 1: Measured performance and solution quality of decision tree, linear classifier, and nearest neighbor.

all MNIST data. The linear classifier is somewhat slower but still very fast relative to the nearest neighbor. Initialization time for decision trees and linear classification depends heavily on the parameters used. In my tests, I saw a clear tradeoff between initialization speed and solution quality. I was able to tune parameters to get good results in under 3 minutes for the decision tree and under 5 minutes for the linear classifier.

One of the most important aspects is the quality of the results. The tests showed clear differences. The decision tree plateaued at a score of ca. 0.85. The Linear classifier performed notably better, reaching scores of 0.91. As could have been predicted, the nearest neighbor agent produced by far the highest score outputting a perfect 1.0 for training images and 0.97 for test images. The perfect training score is self-evident, as it will simply find the original image it is classifying. One last important measure to consider is the memory usage. The linear classifier has very low memory usage as it only stores one weight matrix of size 784×10 . The decision tree uses more memory, although the exact amount depends strongly on the parameters used when generating the tree. It will rarely exceed a few megabytes in this case. Nearest neighbor requires storing the entire MNIST training data ($> 100\text{MB}$) in memory at all times during classification.

4 Conclusion

Out of the three agents, the linear classifier appears the most promising for solving this kind of problem. The decision tree’s accuracy is too low in comparison, and the execution time of nearest neighbor is unrealistic. Both the linear classifier and the decision tree allow a much finer level of heuristics and pruning than the nearest neighbor algorithm. If high accuracy is of the essence, however, nearest neighbor produced the highest quality results.

²Initialization refers to either training of the agent or setup of its data. Since IO is not included in the measurement, initialization time for nearest neighbor is essentially 0.